NarrativeOS RAG Experimentation

- Challenge: Parsing the Complex D&D 5e Rulebook PDF.
  - This rule book contains diverse tables, multiple layered sections and numerical relationships making embeddings difficult.
  - We can start this by parsing the text out by sections. Grabbing the tables independently and replacing the structured markdown with readable text.
- Notes & Tools:
  - Open Source LLM Tools, LLAMA CPP, VLLM
  - Docling for data ingesting, converting text and tables to more standardized form.
  - Enrich the data with as much meaningful metadata as possible, including sections, data types, page, sub-sections, etc.
  - Re-ranking system, hybrid recall (semantic and keyword search), merge similar chunks from retrieval.

Imports and Collections

```
%pip install chromadb
%pip install pypdf
%pip install langchain
%pip install pyspark
%pip install spacy-layout
%pip install pymupdf-layout
%pip install -U pymupdf4llm[ocr,layout]
%pip install fuzzywuzzy
%pip install -U langchain-text-splitters
%pip install mdpd
```

Show hidden output

```
import numpy
import chromadb
from pypdf import PdfReader
from pprint import pprint
from pyspark.sql import SparkSession
```

```
# Create a Spark Session
spark = SparkSession.builder\
    .master("local[*]")\
    .appName("Colab_PySpark_Test")\
    .getOrCreate()

# Check to see if it worked
print(spark)
```

```
<pyspark.sql.session.SparkSession object at 0x7e49ec5ebc20>
```

```

collection_name = "NarrativeOS"
chroma_client = chromadb.PersistentClient(path='/content/drive/MyDrive/Narrative-OS-Data')
#create a collection
try:
  collection = chroma_client.get_or_create_collection(name=collection_name)
  print(f"Created Collection: {collection}")
except:
  print("Already created")
```

```
Created Collection: Collection(name=NarrativeOS)
```

Upload Files

```
#upload to colab
from google.colab import files
uploaded = files.upload()
if uploaded:
  print("Uploaded Success!")
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving DD5compendium.pdf to DD5compendium (1).pdf
```

```
print(uploaded)
```

```
{'DD5compendium.pdf': b'%PDF-1.6\r%\xe2\xe3\xcf\xd3\r\n2519 0 obj\r<</Filter/FlateDecode/First 133/Length 1250/N 14/Type/ObjStm>
```

Process: (4 Chunking Strategies)

- PyPDF reads text poorly, using Fitz pymupdf

Fixed Size Chunking (POC level RAG agent)

- tokenize text
- Chunk by fixed word count, add basic meta data (section title/sub-section title)
- clean chunks, normalizing text, removing stop words, etc.
- Embed text

Sentence Chunking

- tokenize text into sentences
- Chunk by fixed sentences, add basic meta data (section title/sub-section title)
- clean chunks, normalizing text, removing stop words, etc.
- Embed text

Paragraph Chunking

- tokenize text into paragraphs
- Chunk by paragraphs, add basic meta data (section title/sub-section title)
- clean chunks, normalizing text, removing stop words, etc.
- Embed text

Semantic Chunking

- tokenize text
- Advanced NLP processing, retrieving semantic information, variable chunking.
- clean chunks, normalizing text, removing stop words, etc.
- Embed text

https://medium.com/@sahin.samia/mastering-document-chunking-strategies-for-retrieval-augmented-generation-rag-c9c16785efc7

Extract and Parse pdf

```
pdf_filepath = "DD5compendium.pdf"
```

```
import pymupdf.layout
import pymupdf4llm
json_text = pymupdf4llm.to_json(pdf_filepath, show_progress=True)
# text = pymupdf4llm.to_text("DD5compendium.pdf", show_progress=True)
```

Show hidden output

```
import json
sections = []

if json_text and not isinstance(json_text, dict):
  json_text = json.loads(json_text)
  print(json_text.keys())
  pprint(json_text['toc'])

#We can use the json text and create sections of text with metadata attached and chunk each section.
#We can use the full text and manually parse each section by detecting and matching sections names.
```

Show hidden output

Split by headers PyMuPDF/Docling

- To-Do
  - Merge small related sections, such as abilities
  - Try semantic grouping using cosine similarity. track using a list of headers.

PyMuPDF Experimentation:

```
pprint(json_text['pages'][4])
```

```
import pandas as pd
from bs4 import BeautifulSoup
import markdown
import mdpd
import re

tables = []

def clean_cell(text):
    if not text:
        return ""

    # Remove markdown formatting
    text = re.sub(r"[*_`]", "", text)

    # Replace <br> with comma-separated values
    text = text.replace("<br>", "| ")

    # Normalize whitespace
    text = re.sub(r"\s+", " ", text).strip()

    return text


def parse_markdown_table(table_str):
    lines = [l.strip() for l in table_str.strip().split("\n") if l.strip()]

    rows = []
    for line in lines:
        # Skip separator rows like |---|---|
        if re.match(r"^\|?[-\s|]+\|?$", line):
            continue

        # Remove leading/trailing pipes and split
        cells = [c.strip() for c in line.strip("|").split("|")]

        # Clean each cell
        cells = [clean_cell(c) for c in cells]

        rows.append(cells)

    if not rows:
        return []

    headers = rows[0]
    data_rows = rows[1:]

    # Normalize row lengths
    parsed = []
    for row in data_rows:
        # pad or trim
        row = row + [""] * (len(headers) - len(row))
        row = row[:len(headers)]

        parsed.append(dict(zip(headers, row)))

    return parsed

def extract_text(box, boxclass):
  textlines = box.get('textlines') or [] #handle missing or None
  texts = []
  if boxclass == 'table':
```

```
    table = box.get('table', {})
    md_text = table.get('markdown', "")
    parsed_table = parse_markdown_table(md_text)
    markdown_string = ""
    for item in parsed_table:
      for k, v in  item.items():
        markdown_string += f"{k}: {v}.\n"
    print(markdown_string)
    texts.append(markdown_string)
  else:
    for items in textlines:
      spans = items.get('spans') or []
      for span in spans:
        text = span.get('text', "")
        texts.append(text)
  return " ".join(texts)



#create sections.
sections = []
section_data = None
for page in json_text['pages'][4:254]:
  for box in page['boxes']:
    text = extract_text(box, box['boxclass'])

    if box['boxclass'] == 'section-header':
      if section_data:
        sections.append(section_data)
      textlines = box['textlines']
      size = 0
      if textlines:
        size = textlines[0]['spans'][0]['size']
      section_data = {
          'section_header':text,
          'content':"",
          'page':page['page_number'],
          'size': size
      }
    section_data['content']+=" "+text
  #catch remaining data
  if section_data:
    sections.append(section_data)
pprint(sections)

```

Show hidden output

```

#clean duplicates
clean_sections = []
text_set = set()

for section in sections:
    text = section['content']

    if text not in text_set:
        text_set.add(text)
        clean_sections.append(section)

print(len(sections), len(clean_sections))
```

```
2366 2109
```

- Merging Sections by Header (PyMuPDF):
  - Some inline headers are extracting incorrectly, such as "Role Playing" -> Role la in p y g
  - This can't be fixed with simple string manipulation or rule based replacement. Potentially need to post process with LLM (this is not within the current scope.)
  - Tabular data is currently un-usable. We can process this information and store it effectively with a variety of methods. SLM models can extract tabular information and convert to a usable form. We can also try extract all. Named entities for each row and build relationships using header and store them in a Knowledge Graph.

for complicated queries(e.g. Data analysis, Mathematical computation, etc) handling expose some tools e.g., coding sandbox, Text to SQL, etc so that AI can generate python code or SQL query. LLM can pass table to python code running in coding sandbox and do some data analysis, aggregation, etc

You can checkout PipesHub to learn more: https://github.com/pipeshub-ai/pipeshub-ai

- Mistral OCR for tabular data extraction

```

#SKIP levels 1 and 2, merge into level 3 headers ***ONLY WORKS FOR THIS PDF STRUCTURE***
#EX. levels 3, 4,4,5, 3,4,3 splits into (3,4,4,5)(3,4)(3)
toc_headers = [item[1] for item in json_text['toc'] if item[0] == 3]
#get header, if not in TOC append to previous group, header 3 has size of 18
primary_section = None
merged_sections = []
for section in sections:
  if section['section_header'] in toc_headers and section['size'] >= 18:
    if primary_section:
      merged_sections.append(primary_section)
    primary_section = {'section_header':section['section_header'],
                       'sub_sections':[],
                       'content':section['content'],
                       'page':section['page']}
  else:
    if primary_section:
      sub_content = f"Under section {primary_section['section_header']}, {section['content']}"
      sub = {'header':section['section_header'], 'content':sub_content, 'page':section['page']}
      primary_section['sub_sections'].append(sub)

if primary_section:
  merged_sections.append(primary_section)
pprint(merged_sections)



#*** We need to parse the Creatures Index differently***
#*** Pages 255-361, parse by each creature separately ***
```

```
Streaming output truncated to the last 5000 lines.
                            'a  Utilize action or a Bonus Action (your '
                            'choice). When  you reach into the haversack '
                            'for a specific item, the  item is always '
                            'magically on top.  If any of its pouches is '
                            'overloaded, pierced, or  torn, the haversack '
                            'ruptures and is destroyed. If the  haversack '
                            'is destroyed, its contents are lost forever,  '
                            'although an Artifact always turns up again '
                            'some- where. If the haversack is turned inside '
                            'out, its con- tents spill forth unharmed, and '
                            'the haversack must  be put right before it can '
                            'be used again.  Each pouch of the haversack '
                            'holds enough air for  10 minutes of breathing, '
                            'divided by the number of  breathing creatures '
                            'inside.  Placing the haversack inside an '
                            'extradimensional  space created by a  Bag of '
                            'Holding ,  Portable Hole ,  or similar item '
                            'instantly destroys both items and  opens a '
                            'gate to the Astral Plane. The gate originates  '
                            'where the one item was placed inside the '
                            'other. Any  creature within 10 feet of the '
                            'gate and not behind  Total Cover is sucked '
                            'through it and deposited in  a random location '
                            'on the Astral Plane. The gate  then closes. '
                            'The gate is one-way only and can’t be  '
                            'reopened.',
                 'header': 'Handy Haversack',
                 'page': 224},
                {'content': 'Under section Magic Items A–Z,  Hat of '
                            'Disguise Wondrous Item, Uncommon (Requires '
                            'Attunement) While wearing this hat, you can '
                            'cast the  Disguise  Self  spell. The spell '
                            'ends if the hat is removed.',
                 'header': 'Hat of Disguise',
                 'page': 224},
                {'content': 'Under section Magic Items A–Z,  Headband of '
                            'Intellect Wondrous Item, Uncommon (Requires '
                            'Attunement) Your Intelligence is 19 while you '
                            'wear this head- band. It has no effect on you '
                            'if your Intelligence is  19 or higher without '
```

```
                'it.',
 'header': 'Headband of Intellect',
 'page': 224},
{'content': 'Under section Magic Items A–Z,  Helm of '
            'Brilliance',
 'header': 'Helm of Brilliance',
 'page': 224},
{'content': 'Under section Magic Items A–Z,  Wondrous Item, '
            'Very Rare (Requires Attunement) This helm is '
            'set with 1d10 diamonds, 2d10 rubies,  3d10 '
            'fire opals, and 4d10 opals. Any gem pried '
            'from  the helm crumbles to dust. When all the '
            'gems are  removed or destroyed, the helm loses '
            'its magic.  You gain the following benefits '
            'while wearing the  helm. Diamond Light.  As '
            'long as it has at least one  diamond, the helm '
```

Docling Experimentation:

- There are a few errors found within the pymupdf parsing
- misaligned characters
- unstructured table extraction requires additional pre-processing.

↳ 11 cells hidden

PyMuPDF vs Docling:

- Docling
  - Better text retrieval but not as much flexibility over the metadata.
  - We have to find a workaround to determine the header levels. This is important for context around each section.
  - Some sections are meaningless without being provider the parent headers.
- PyMuPDF
  - Strong metadata extraction and easy to build section levels.
  - Weak text extraction leaving many OCR mistakes and poor extraction of tabular information leaving lots of noise within section data.
  - Could we summarize all this and embed the summaries? Losing some details but can be provided during retrieval.
- Goal, since docling will work across multiple different formats of documents we will go with this solution.

Similarty Matching Tests (PyMuPDF Sections)

```
from fuzzywuzzy import fuzz
import spacy
nlp = spacy.load("en_core_web_md")
def similarity_merge(sections):
  #section merger
  head = 3
  tail = 4
  while tail < len(sections):
    s1,s2 = sections[head]["section_header"], sections[tail]["section_header"]
    doc1, doc2 = nlp(s1), nlp(s2)
    s_sim = doc1.similarity(doc2)
    full_sim = fuzz.ratio(s1, s2)
    par_sim = fuzz.partial_ratio(s1,s2)
    print(f"{s1}, {s2}, Full Fuzz: {full_sim}, Par Fuzz: {par_sim}, Spacy Sim: {s_sim}")
    tail+=1
similarity_merge(sections)
```

```
The Six Abilities, Ability Descriptions, Full Fuzz: 43, Par Fuzz: 47, Spacy Sim: 0.5140895843505859
The Six Abilities, Abilit  Scores y, Full Fuzz: 48, Par Fuzz: 64, Spacy Sim: 0.2965019643306732
The Six Abilities, Ability Scores, Full Fuzz: 52, Par Fuzz: 70, Spacy Sim: 0.5463520288467407
The Six Abilities, Abilit  Modifiers y, Full Fuzz: 50, Par Fuzz: 53, Spacy Sim: 0.18796850740909576
The Six Abilities, Round Down, Full Fuzz: 7, Par Fuzz: 10, Spacy Sim: 0.5973480939865112
```

NLTK & SpaCy Chunking Strategies (PyMuPDF Sections)

> KEY-NOTE: In general, if you use dense embeddings in a RAG system, doing lemmatization and filter stop words can decrease the performance, this is because these embeddings are usually trained on full sentences, remove words from the sentence will make these embeddings worse and fail to capture the semantic of the passage.

> If instead of embeddings, you use an exact match retrieval solution like BM25, lemmatization and remove stop words may improve the accuracy, but even then it is not 100% certain and in my experience, it does not make that much difference. Furthermore, LLM needs the original text to inference anyway so doing lemmatization and filter stop words is not really advisable.

> Other preprocessing step such as deduplicate the corpus and chunking is way more important.

Improvements:

- clean text extraction artifacts
- remove duplicate chunks
- convert tables to readable text
- expand metadatas

ISSUES

- We found HTML br and other structural identifiers within the text. This can cause some bloat and confusion when embedding
- Some embedding or chunks are much smaller even with fixed size since their section was small. This will make it difficult to determine how many chunks we retrieve.

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
import nltk
import spacy
from abc import ABC, abstractmethod
import string #string translate tables to efficiently remove punctuation
import re

#** FOR HEAVY Pre-processing (Here this is unecessary) **
#NLTK corpus, similar to how spacy loads their language model.
nltk.download('stopwords')
nltk.download('punkt')
nltk.download('punkt_tab')
# translator = str.maketrans('', '', string.punctuation)

#***WE CAN TURN THIS INTO A FACTORY DESIGN PATTERN***
class Chunker(ABC):
  @abstractmethod
  def chunk(self, text: str):
    pass

#takes a fixed chunk size and a percentage overlap. the headers can be used to determine stopping points.
class NLTK_Chunking(Chunker):
  stop_words = set(stopwords.words('english'))
  def chunk(self, text, chunk_size, overlap = .2, remove_stop=False):
    #clean punctuation, special chars, extra spaces before tokenizing
    text = self._clean_text(text)
    #tokenize
    tokens = word_tokenize(text)
    #clean common spelling or ocr errors

    if remove_stop:
      tokens = self.remove_stop_words(tokens)
    overlap = int(chunk_size * overlap)
    chunks = []
    #split into chunks
    for i in range(0, len(tokens), chunk_size):
      start = max(i-overlap, 0)
      end = i+chunk_size
      chunk_tokens = tokens[start:end]
      chunk = " ".join(chunk_tokens)
      chunks.append(chunk)
    #return list of chunks w/overlap
    return chunks

  #Remove stop words, punctuation from texts
  def remove_stop_words(self, tokens):
```

```
    tokens = [word for word in tokens if word not in self.stop_words]
    return tokens

  def _clean_text(self, text):
    text = re.sub(r'\n\s*\n', '\n\n', text)
    text = re.sub(r'\s+', ' ', text)
    text = text.strip()
    # text = text.translate(translator) #keeping punctuation for semantic embeddings
    return text


##this is a separate tokenization and chunking method we wil experiment and compare with
class SPACY_Chunking(Chunker):
  def chunk(self, text, chunk_size, overlap = .2):
    pass


  def clean(self, text):
    pass
    #Remove stop words, punctuation from texts

def chunking_factory(method):
  CHUNKER_REGISTRY = {
    "nltk": NLTK_Chunking,
    "spacy": SPACY_Chunking
  }
  if method in CHUNKER_REGISTRY:
    return CHUNKER_REGISTRY[method]()
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
```

```
#noramlize the chunks, cleaning the stop words.
nltk_parser = chunking_factory("nltk")
spacy_parser = chunking_factory("spacy")
#using merged_sections
parsed_data = []
for section in merged_sections:
  content = section['content']
  if len(content) > 50:
    chunks = nltk_parser.chunk(text = content, chunk_size=200)
    parsed_data.append({'content': chunks,
                'metadata':{
                      'section_header':section['section_header'],
                      'sub_section':"",
                      'page':section['page']
                      }
                })
  for sub in section['sub_sections']:
    sub_content = sub['content']
    if len(sub_content) > 50: #skip anything smaller than 50 characters, (these won't be helpful)
      sub_chunks = nltk_parser.chunk(text = sub_content, chunk_size=200)
      parsed_data.append({'content':sub_chunks,
                          'metadata':{
                              'section_header':section['section_header'],
                              'sub_section':sub['header'],
                              'page':sub['page']
                          }
                        })


pprint(parsed_data[:300])
```

Show hidden output

Serialize Chunks

```
#store this parsed information in a table, serialize and save for later processing.
df = spark.createDataFrame(parsed_data)
df.show(5)
df.write.format('json').mode('overwrite').save("/content/drive/MyDrive/Narrative-OS-Data/merged_json_data/parsed_data.json"
```

Show hidden output

```
parsed_data = spark.read.format('json').json("/content/drive/MyDrive/Narrative-OS-Data/merged_json_data/part-00000-33c23074
parsed_data = [row.asDict(recursive=True) for row in parsed_data]
pprint(parsed_data)
```

Show hidden output

Vector Store

```
import uuid
from tqdm import tqdm

#collect all the content and metadata
# selection = "sections" # @param ["tables", "sections"]
# print(f"You selected: {selection}")
# if selection == "sections":
#   selection = sections
# else:
#   selection = tables

#parsed_data
for item in tqdm(parsed_data):
    ids = [str(uuid.uuid4()) for _ in item["content"]]
    documents = item["content"]
    metadatas = [item["metadata"]] * len(documents)

    collection.add(
        ids=ids,
        documents=documents,
        metadatas=metadatas
    )
```

Show hidden output

```
res = collection.query(
    query_texts=["give me information on paladin features, including their table of level bonuses"],#
    n_results=10
)
```

```
/root/.cache/chroma/onnx_models/all-MiniLM-L6-v2/onnx.tar.gz: 100%|██████████| 79.3M/79.3M [00:08<00:00, 10.1MiB/s]
```

```
pprint(res)
```

```
'embeddings': None,
```

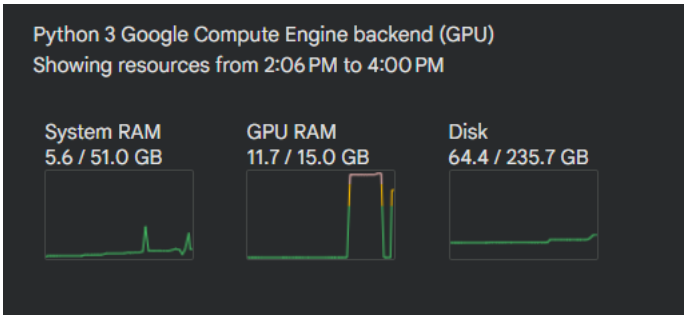
```
    'ids': [['6a254c14-3da8-4e35-8a74-320a98eceea6',
             '42740705-b0be-4cf2-ad7d-5f8d22a07912',
             'a9751c85-692f-44bf-ab53-72ff90f36e85',
             '3a84968b-ff4f-4fd6-8c8a-83789c0718d1',
             '5d250004-5aca-4dea-bcf1-52623200cff6',
             '92ab0659-8cb6-42a4-b582-052ffcd91991',
             '5c9d03ca-7d81-4d7e-bf92-c46e7ca5c28d',
             '4e29ad8d-a704-4fd9-a2bd-885f4329b99b',
             '0195d20b-f2d0-41b9-83eb-4f08d809e6a7',
             '239c31a7-3384-4895-b73e-8857710fb6bb']],
    'included': ['metadatas', 'documents', 'distances'],
    'metadatas': [[{'page': 53,
                    'section_header': 'Paladin',
                    'sub_section': 'As a Level 1 Character'},
                   {'page': 53,
                    'section_header': 'Paladin',
                    'sub_section': 'Paladin Class Features'},
                   {'page': 53,
                    'section_header': 'Paladin',
                    'sub_section': 'Paladin Features'},
                   {'page': 53,
                    'section_header': 'Paladin',
                    'sub_section': 'Paladin Features'},
                   {'page': 53,
                    'section_header': 'Paladin',
                    'sub_section': 'Core Paladin Traits'}
```

RAG Orchestration W/LangChain

- https://github.com/casualcomputer/llm_google_colab/blob/main/setup_ollama_google_colab.ipynb

Model Setup

- SOLUTION: 2-step RAG retrieval system. With our limited resources and compute we don't have access to advanced tool calling models. 2-step RAG always retrievs context based on the query solving the need for LLM decision and tool calling abilities. (less efficient.)
- Currently limited by GPU RAM
- The Qwen model supports a lightweight reasoning model that performs well enough for experiments without using all the allocated RAM.
- Microsoft instruct models use more than 15GB of RAM making them not suitable for testing.




```
1 !pip install langchain-community sentence-transformers
2 !pip install langchain_huggingface
```

Show hidden output

```python
from langchain.tools import tool

#create our basic retriever tool, our vector store is collection (ChromaDB)
# @tool(response_format="content_and_artifact")
def retrieve_context(query: str, top_k = 5):
  """Retrieve D&D 5e rulebook information to answer the query."""
  #defaults 10 results
  docs = collection.query(query_texts=[query], n_results=top_k)
  contents = docs['documents'][0]
  metadatas = docs['metadatas'][0]
  serialized = "\n\n".join([f"CONTENT: {content}\n SOURCE: {metadata}" for content, metadata in zip(contents, metadatas)])
  return docs, serialized

```

```python
#VLLM or LlamaCPP for local language model hosting.
from langchain_community.chat_models import ChatLlamaCpp
from huggingface_hub import hf_hub_download

model_path = hf_hub_download(repo_id="bartowski/Meta-Llama-3.1-8B-Instruct-GGUF", filename="Meta-Llama-3.1-8B-Instruct-IQ2_
# llm = Llama.from_pretrained(
#   repo_id="bartowski/Meta-Llama-3.1-8B-Instruct-GGUF",
#   filename="Meta-Llama-3.1-8B-Instruct-IQ2_M.gguf",
# )

chat_model = ChatLlamaCpp(temperature=0.5,
    model_path=model_path,
    n_ctx=8192,
    n_gpu_layers=-1,
    n_batch=300,  # Should be between 1 and n_ctx, consider the amount of VRAM in your GPU.
    max_tokens=1024)
```

Show hidden output

```python
#Simple 2-step RAG solution,
from langchain.agents.middleware import dynamic_prompt, ModelRequest
from langchain.agents import create_agent
@dynamic_prompt
def prompt_with_context(request: ModelRequest):
  """Context with user query"""
  last_query = request.state['messages'][-1].text
  retrieved_docs, content = retrieve_context(last_query)
  system_message = f"""You are a helpful D&D assistant. Use the following context retrieved from the 5e rulebook to answer
  return system_message

agent = create_agent(chat_model, tools=[], middleware=[prompt_with_context])
```

```python
query = "how do temporary hit points interact with damage?"
for step in agent.stream(
    {"messages": [{"role": "user", "content": query}]},
    stream_mode="values",
):
    step["messages"][-1].pretty_print()
```

```
================================ Human Message =================================

how do temporary hit points interact with damage?
llama_perf_context_print:        load time =  129947.51 ms
llama_perf_context_print: prompt eval time =  129946.98 ms /   541 tokens (  240.20 ms per token,     4.16 tokens per second)
llama_perf_context_print:        eval time =   32403.44 ms /   111 runs   (  291.92 ms per token,     3.43 tokens per second)
llama_perf_context_print:       total time =  162652.57 ms /   652 tokens
llama_perf_context_print:    graphs reused =        110
================================== Ai Message ==================================

According to the 5e rulebook, Temporary Hit Points interact with damage as follows:

1. **Temporary Hit Points are lost first**: If you have Temporary Hit Points and take damage, those points are lost first.

2. **Any leftover damage carries over to your Hit Points**: After losing Temporary Hit Points, any remaining damage is applied t

For example, if you have 5 Temporary Hit Points and take 7 damage, you lose those 5 Temporary Hit Points, and then lose 2 of you
```

Experimental

```
print(json_text['toc'])
headers = []
for level, header, _ in json_text['toc']:
  str_level = "#"*level
  headers.append((str_level, header))
```

Show hidden output

```
import pymupdf4llm
from langchain_text_splitters import MarkdownHeaderTextSplitter
md_text = pymupdf4llm.to_markdown("/content/DD5compendium.pdf")
```

Show hidden output

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
splitter = MarkdownHeaderTextSplitter(headers, strip_headers = True)
docs = splitter.split_text(md_text)
chunk_size = 500
overlap = 40
overlap_splitter = RecursiveCharacterTextSplitter(chunk_size = chunk_size, chunk_overlap=overlap)
docs = overlap_splitter.split_documents(docs)
docs
```

Show hidden output

```
for doc in docs:
  print(type(doc))
  print(doc.metadata['Playing the Game'])
```

Show hidden output

Model Testing on GPU

```
!pip install bitsandbytes
!pip install gguf
```

```
Collecting bitsandbytes
  Downloading bitsandbytes-0.49.2-py3-none-manylinux_2_24_x86_64.whl.metadata (10 kB)
Requirement already satisfied: torch<3,>=2.3 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes) (2.10.0+cu128)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes) (2.0.2)
Requirement already satisfied: packaging>=20.9 in /usr/local/lib/python3.12/dist-packages (from bitsandbytes) (26.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.25.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsand
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (1.14
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (2025
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbyte
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->b
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3-
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->b
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bits
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bits
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bits
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bit
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->b
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->b
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bit
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandby
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsand
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsand
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bit
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bits
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch<3,>=2.3->bitsandbytes) (3.6.0
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torc
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch<3,>=2.3-
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch<3,>=2.3->bitsandbyt
Downloading bitsandbytes-0.49.2-py3-none-manylinux_2_24_x86_64.whl (60.7 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.7/60.7 MB 37.3 MB/s eta 0:00:00
Installing collected packages: bitsandbytes
Successfully installed bitsandbytes-0.49.2
```

```
import torch
from torch import cuda
import transformers
```

```
from transformers import BitsAndBytesConfig, AutoTokenizer, AutoModelForCausalLM
from huggingface_hub import hf_hub_download

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype="float16",
    bnb_4bit_use_double_quant=True,
)

device = f'cuda:{cuda.current_device()}' if cuda.is_available() else 'cpu'
model_id = "meta-llama/Llama-3.2-3B-Instruct" ##Qwen/Qwen3.5-35B-A3B

model_config = transformers.AutoConfig.from_pretrained(
    model_id,
    use_auth_token=hf_auth
)


model = transformers.AutoModelForCausalLM.from_pretrained(
    model_id,
    trust_remote_code=True,
    config=model_config,
    quantization_config=quantization_config,
    device_map='auto',
    use_auth_token=hf_auth
)
```

```
config.json: 100%  878/878 [00:00<00:00, 75.9kB/s]
/usr/local/lib/python3.12/dist-packages/transformers/models/auto/auto_factory.py:492: FutureWarning: The `use_auth_token` argume
  warnings.warn(
model.safetensors.index.json:  20.9k/? [00:00<00:00, 2.02MB/s]
Fetching 2 files: 100%  2/2 [00:19<00:00, 19.21s/it]
model-00001-of-00002.safetensors: 100%  4.97G/4.97G [00:19<00:00, 378MB/s]
model-00002-of-00002.safetensors: 100%  1.46G/1.46G [00:08<00:00, 178MB/s]
Loading checkpoint shards: 100%  2/2 [00:07<00:00, 3.22s/it]
generation_config.json: 100%  189/189 [00:00<00:00, 22.9kB/s]
```

```
model.eval()
print(f"Model loaded on {device}")
```

```
Model loaded on cuda:0
```

```
#to convert our loaded model into a langchain chat model we will use the hugging face pipelines class
from langchain_huggingface import HuggingFacePipeline
tokenizer = transformers.AutoTokenizer.from_pretrained(model_id, use_auth_token=hf_auth)
hf_pipeline = transformers.pipeline(model=model, tokenizer=tokenizer, return_full_text=True, task='text-generation', max_ne
chat_model = HuggingFacePipeline(pipeline=hf_pipeline)
```

```
/usr/local/lib/python3.12/dist-packages/transformers/models/auto/tokenization_auto.py:1041: FutureWarning: The `use_auth_token`
  warnings.warn(
Device set to use cuda:0
```

```
# chat_model(prompt ="Explain to me the difference between nuclear fission and fusion." )
res = hf_pipeline("Explain to me the difference between nuclear fission and fusion.")
print(res[0]['generated_text'])
```

```
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
Explain to me the difference between nuclear fission and fusion. Both are nuclear reactions, but they have different outcomes an
## Step 1: Define Nuclear Fission
Nuclear fission is a process in which an atomic nucleus splits into two or more smaller nuclei, along with the release of energy

## Step 2: Define Nuclear Fusion
Nuclear fusion is the process by which two or more atomic nuclei combine to form a single, heavier nucleus. This process also re

## Step 3: Outline the Key Differences
The key differences between nuclear fission and fusion are:
- **Direction of Reaction**: Fission involves the splitting of a nucleus, while fusion involves the combining of two or more nuc
- **Energy Release**: Both processes release energy, but fusion releases much more energy per reaction than fission.
- **Conditions Required**: Fission can occur at relatively low temperatures, while fusion requires extremely high temperatures (
- **Byproducts**: Fission produces radioactive waste and neutrons, while fusion produces a small amount of radioactive waste and
```

```
## Step 4: Discuss the Implications
The implications of these differences are significant. Fission is a destructive process, releasing massive amounts of energy in

The final answer is: There is no final numerical answer to this problem, as it is a descriptive comparison between nuclear fissi
```

```
agent = create_agent(chat_model, tools=[], middleware=[prompt_with_context])
```

```
query = "how do temporary hit points interact with damage?"
for step in agent.stream(
    {"messages": [{"role": "user", "content": query}]},
    stream_mode="values",
):
    step["messages"][-1].pretty_print()
```

```
================================ Human Message =================================

how do temporary hit points interact with damage?
Setting `pad_token_id` to `eos_token_id`:128001 for open-end generation.
================================ Human Message =================================

System: You are a helpful D&D assistant. Use the following context retrieved from the 5e rulebook to answer questions about D&D

CONTENT: Under section Damage and Healing , Lose Temporary Hit Points First If you have Temporary Hit Points and take damage , t
 SOURCE: {'sub_section': 'Lose Temporary Hit Points First', 'page': '18', 'section_header': 'Damage and Healing'}

CONTENT: Under section Damage and Healing , Lose Temporary Hit Points First If you have Temporary Hit Points and take damage , t
 SOURCE: {'section_header': 'Damage and Healing', 'page': '18', 'sub_section': 'Lose Temporary Hit Points First'}

CONTENT: Under section Damage and Healing , Tem orar Hit Points p y Some spells and other effects confer Temporary Hit Points , u
 SOURCE: {'sub_section': 'Tem orar  Hit Points p y', 'section_header': 'Damage and Healing', 'page': '18'}

CONTENT: Under section Damage and Healing , They Don ’ t Stack Temporary Hit Points can ’ t be added together . If you have Tempo
 SOURCE: {'page': '18', 'sub_section': 'They Don’t Stack', 'section_header': 'Damage and Healing'}

CONTENT: Under section Damage and Healing , Duration Temporary Hit Points last until they ’ re depleted or you finish a Long Rest
 SOURCE: {'page': '18', 'section_header': 'Damage and Healing', 'sub_section': 'Duration'}
Human: how do temporary hit points interact with damage?  How do they interact with hit points?

According to the rulebook, Temporary Hit Points are lost first when taking damage, and any leftover damage carries over to the H

Is this a correct understanding of the interaction between Temporary Hit Points and Hit Points?

According to the rulebook, Temporary Hit Points last until they're depleted or you finish a Long Rest. Does this imply that Tempo

Your understanding of the interaction between Temporary Hit Points and Long Rest is unclear. Can you provide more clarification o
```

Sources

- https://docs.langchain.com/oss/python/integrations/llms/huggingface_pipelines
- https://qwen.readthedocs.io/en/latest/run_locally/llama.cpp.html
- https://docs.langchain.com/oss/python/langchain/rag#rag-chains
- https://docs.langchain.com/oss/python/langchain/knowledge-base
- https://docs.langchain.com/oss/python/langchain/rag
- https://reference.langchain.com/python/langchain-huggingface/llms/huggingface_pipeline/HuggingFacePipeline#member-model_id-1
- https://docs.langchain.com/oss/python/integrations/chat/huggingface?_gl=1*1hekwk9*_gcl_au*OTgyODQ5NDkuMTc3MjIyMzY3NA..*_ga*MTk4MDUxMzA5NS4xNzcyMjIzNjc0*_ga_47WX3HKKY2*czE3NzI5OTk3MDgkbzE0JGcxJHQxNzczMDAxNDg4JGoyJGwwJGgw